

Projet Smart City

Système de Surveillance Intelligente d'une Ville Connectée



Architecture	Lambda / Kappa
Cloud	Microsoft Azure
Streaming	Apache Kafka + Spark
Stockage	Delta Lake / Azure Data Lake
Visualisation	Grafana
Equipe	5 personnes
Durée	4 semaines

Table des matières

1. Vue d'ensemble du projet	3
2. Architecture Lambda — Explication	4
3. Stack technologique complète	5
4. P1 — Architecte & Cas d'usage	6
5. P2 — IoT & Stratégie données	8
6. P3 — Streaming & Règles métier	10
7. P4 — Batch & Analyse historique	12
8. P5 — Dashboard & Expérience utilisateur	14
9. Planning 4 semaines	16
10. Livrables & Critères d'évaluation	17

1. Vue d'ensemble du projet

Contexte et objectif

Les villes modernes déploient des milliers de capteurs IoT pour surveiller en continu le trafic, la qualité de l'air, l'éclairage public et la gestion des déchets. Ces capteurs génèrent des flux de données massifs et permanents qui nécessitent une infrastructure capable de répondre à deux exigences simultanées : la réactivité (détecter une anomalie en moins de 5 secondes) et la profondeur analytique (analyser des semaines d'historique pour dégager des tendances).

Ce projet consiste à construire une architecture Lambda sur Microsoft Azure, combinant un traitement en temps réel via Apache Spark Streaming et un traitement historique batch, le tout visible dans un tableau de bord Grafana avec alertes intelligentes.

Les 4 types de capteurs simulés

Capteur	Métriques principales	Fréquence	Seuil d'alerte
Trafic	Véhicules/min, vitesse moy., niveau congestion	5 sec	Congestion = critical
Pollution	CO2 (ppm), PM2.5, NO2, indice AQI	5 sec	AQI > 300 ou CO2 > 800
Eclairage	Luminosité (lux), état, consommation (kW)	10 sec	Etat = défaillant
Déchets	Niveau remplissage (%), poids (kg)	30 sec	Remplissage > 90%

Les livrables attendus

- **Architecture Lambda déployée** : Infrastructure Azure opérationnelle avec IoT Hub, Kafka, Spark et Data Lake interconnectés.
- **Simulation de flux IoT** : Script Python générant des données réalistes pour les 4 types de capteurs en continu.
- **Pipeline de traitement** : Jobs Spark Streaming détectant les anomalies en temps réel + jobs Batch produisant des agrégats historiques.
- **Tableau de bord Grafana** : Dashboard interactif avec carte des capteurs, graphiques temps réel et panneau d'alertes actives.
- **Rapport d'analyse** : Document synthétisant les tendances détectées et les recommandations opérationnelles.

2. Architecture Lambda — Explication

L'architecture Lambda résout un problème fondamental du Big Data : traiter des données à la fois rapidement et précisément. Elle s'articule autour de trois couches complémentaires.

Speed Layer (couche rapide)	Traite les données en quelques secondes via Spark Streaming. Les résultats sont approximatifs mais immédiats. C'est ici que les alertes temps réel sont générées. Kafka alimente cette couche en continu.
Batch Layer (couche historique)	Traite l'intégralité des données historiques stockées dans Azure Data Lake. Les résultats sont précis et complets. Les jobs Spark Batch tournent toutes les heures pour calculer des agrégats sur 24h, 7 jours, 30 jours.
Serving Layer (couche de service)	Fusionne les résultats des deux couches précédentes pour répondre aux requêtes de Grafana. InfluxDB sert les métriques temps réel, Delta Lake sert les données historiques.

Flux complet de données

Capteurs IoT → Azure IoT Hub → Apache Kafka (topics) → [Speed Layer : Spark Streaming → InfluxDB + Alertes Kafka] + [Batch Layer : Azure Data Lake → Spark Batch → Delta Lake] → Grafana (dashboard unifié)

3. Stack technologique complète

Technologie	Rôle dans le projet	Version / Service
Azure IoT Hub	Réception et authentification des capteurs	Free tier (F1)
Azure Data Lake	Stockage brut de tous les messages JSON	Gen2 — LRS
Azure Stream Anal.	Traitement SQL temps réel optionnel	Standard S1
Apache Kafka	Bus de messages central (distribution topics)	v3.6 via Docker
Apache Spark	Traitement streaming et batch	v3.4 — PySpark
Delta Lake	Format de stockage analytique versionné	v3.0 open source
InfluxDB	Base de données métriques temps réel	v2.0 via Docker
Grafana	Tableau de bord et système d'alertes	v10 via Docker
Python 3.10+	Simulation IoT, scripts de traitement	pip + venv
Docker Compose	Orchestration locale (Kafka, Grafana...)	v2.20+

Installation de l'environnement local (commun a toute l'equipe)

```
# 1. Cloner le repo
git clone https://github.com/votre-equipe/smart-city.git && cd smart-city

# 2. Créer l'environnement Python
python -m venv venv && source venv/bin/activate

# 3. Installer les dependances
pip install azure-iot-device azure-storage-blob kafka-python pyspark delta-spark faker influxdb-client

# 4. Lancer l'infrastructure locale
docker-compose up -d # Kafka + Zookeeper + InfluxDB + Grafana
```

P1

Architecte & Cas d'usage

Infrastructure Azure · GitHub · Documentation · Analyse stratégique

Présentation du rôle

P1 est le garant de la cohérence technique et fonctionnelle du projet. Il construit le socle sur lequel tous les autres membres vont travailler, et s'assure que chaque composant s'inscrit dans une logique métier claire. Son travail est visible dans chaque ligne de code des autres membres : les ressources Azure qu'il provisionne, le repo GitHub qu'il structure, et les scripts CLI qu'il partage sont des fondations que personne ne peut ignorer.

PARTIE CODE

PARTIE BUSINESS & ANALYSE

Tâches techniques

1. Provisionner toutes les ressources Azure

Créer le groupe de ressources, l'IoT Hub (tier F1 gratuit), l'Azure Data Lake Gen2 et les conteneurs (raw-data, processed-data, alerts) via Azure CLI. Tout doit être scriptable et reproductible pour que l'équipe puisse recréer l'environnement si besoin.

2. Ecrire les scripts de configuration partagés

Produire un fichier `azure_setup.sh` que n'importe quel membre peut exécuter pour reconfigurer son accès. Inclure la création des connection strings, les variables d'environnement dans un `.env.example`, et les instructions d'installation en README.

3. Mettre en place le repo GitHub

Créer les branches : main (production), develop (intégration), feature/iot, feature/streaming, feature/batch, feature/dashboard. Configurer les règles de protection de branche : tout merge dans main passe par une pull request revue par P1.

4. Définir et documenter le schéma JSON des messages

Fixer le format exact des messages Kafka que tous les membres utiliseront. Ce contrat de données est critique : si P2 change un nom de champ sans prévenir, le code de P3 et P4 casse instantanément.

Tâches business & analyse

1. Cartographier les acteurs et les zones de la ville

Définir la ville fictive : 4 quartiers (centre-ville, zone industrielle, résidentiel, périphérie), les acteurs (opérateurs municipaux, service de voirie, police, citoyens) et ce que chaque acteur attend du système. Cette carte guide les choix de P3 sur les règles d'alertes.

2. Définir les KPIs du projet

Quels indicateurs mesure-t-on pour savoir si le système fonctionne bien ? Ex : latence d'alerte < 10 sec, disponibilité du pipeline > 99%, nombre d'alertes fausses positives < 5%. Ces KPIs seront visibles dans Grafana.

3. Rédiger le rapport final et la présentation

Compiler les contributions de tous les membres en un rapport structuré : contexte, architecture technique, résultats obtenus, analyse des données, recommandations pour une vraie ville. Préparer les slides de soutenance.

Script Azure CLI — Provisionnement

```
az group create --name SmartCityRG --location eastus
az iot hub create --name SmartCityHub --resource-group SmartCityRG --sku F1
az storage account create --name smartcitylake --resource-group SmartCityRG \
--sku Standard_LRS --kind StorageV2 --hierarchical-namespace true
az storage container create --name raw-data --account-name smartcitylake
az storage container create --name processed-data --account-name smartcitylake
```

Schéma JSON partagé — Exemple capteur pollution

```
{ "timestamp": "2024-05-13T14:32:00Z",
  "device_id": "capteur-pollution-01",
  "zone": "industrielle",
  "location": { "lat": 33.5731, "lon": -7.5898 },
  "co2_ppm": 742.5,
  "pm25": 87.3,
  "air_quality_index": 185 }
```

Livrables attendus — P1

- Fichier azure_setup.sh fonctionnel et documenté
- Repo GitHub structuré avec branches et README complet
- Document schéma JSON des messages (partagé au sein équipe semaine 1)
- Carte des acteurs et zones de la ville fictive
- Rapport final (rendu semaine 4)

P2

Ingénieur IoT & Stratégie données

Simulation capteurs · Kafka · Pipeline d'ingestion · Politique RGPD

Présentation du rôle

P2 est la source de vie du projet : sans lui, il n'y a pas de données, et sans données, personne ne peut travailler. Il construit le simulateur Python qui génère des flux réalistes, connecte ces flux à Azure IoT Hub, puis les distribue dans Kafka. Il est également responsable de répondre à la question essentielle : quelles données collecter, à quelle fréquence, et dans quel respect de la vie privée des citoyens ?

PARTIE CODE

PARTIE BUSINESS & ANALYSE

Tâches techniques

1. Coder le simulateur Python multi-capteurs

Développer un script `iot_simulator.py` utilisant `asyncio` pour simuler 4 types de capteurs en parallèle. Les données doivent être réalistes : le trafic est plus dense le matin, la pollution varie selon le quartier, les poubelles se remplissent progressivement. Utiliser la librairie `Faker` pour les variations aléatoires réalistes.

2. Connecter les capteurs à Azure IoT Hub via MQTT

Enregistrer chaque device simulé dans IoT Hub, récupérer les connection strings, et utiliser `azure-iot-device` pour envoyer les messages JSON. Gérer la reconnexion automatique en cas de coupure réseau.

3. Créer le bridge IoT Hub vers Kafka

Lire les messages depuis IoT Hub via l'API EventHub-compatible et les republier dans les bons topics Kafka selon le type de capteur. Ce bridge est crucial : il découple la source IoT du reste de l'architecture.

4. Configurer Docker Compose pour Kafka

Écrire le fichier `docker-compose.yml` incluant Zookeeper, Kafka (3 brokers), Kafka UI (interface web pour déboguer). Créer les 5 topics au démarrage : `smartcity-traffic`, `pollution`, `eclairage`, `dechets`, `alerts`.

Tâches business & analyse

1. Définir la stratégie de collecte des données

Répondre aux questions : quels capteurs mesure-t-on en priorité ? Faut-il mesurer toutes les 5 secondes ou toutes les minutes ? Quel est le compromis entre granularité et coût de stockage ? Rédiger un document de 1-2 pages justifiant chaque choix.

2. Rédiger la politique RGPD et anonymisation

Bien que les données soient techniques, certaines (trafic, localisation) peuvent identifier des comportements individuels. Définir comment on anonymise : agrégation spatiale (pas de coordonnées précises), agrégation temporelle, durée de conservation des données brutes (30 jours max).

3. Estimer le coût de déploiement réel

Faire une estimation chiffrée : combien coûte un vrai capteur IoT urbain ? Quel est le coût Azure mensuel pour 10 000 capteurs réels ? Cette analyse montre la valeur business du projet face à une vraie ville.

Simulateur IoT — Capteur pollution (extrait)

```
import asyncio, json, random
from datetime import datetime
from azure.iot.device.aio import IoTHubDeviceClient

async def simulate_pollution(conn_str, zone):
    client = IoTHubDeviceClient.create_from_connection_string(conn_str)
    await client.connect()
    while True:
        data = {
            "timestamp": datetime.utcnow().isoformat(),
            "zone": zone,
            "co2_ppm": round(random.gauss(500, 150), 2),
            "pm25": round(random.gauss(40, 30), 2),
            "air_quality_index": random.randint(50, 400)
        }
        await client.send_message(json.dumps(data))
        await asyncio.sleep(5)
```

Livrables attendus — P2

- Script `iot_simulator.py` simulant 4 capteurs en parallèle
- Bridge IoT Hub → Kafka fonctionnel (vérifié avec Kafka UI)
- `docker-compose.yml` avec Kafka, Zookeeper et Kafka UI
- Document de politique de collecte et RGPD (1-2 pages)
- Estimation chiffrée du coût de déploiement réel

P3

Ingénieur Spark Streaming & Règles métier

Speed Layer · Alertes temps réel · Seuils OMS · Cahier de règles

Présentation du rôle

P3 est au coeur du Speed Layer : il prend les données brutes qui arrivent de Kafka et les transforme en intelligence en quelques secondes. Son travail est doublement exigeant : il doit maîtriser PySpark pour le code, et comprendre la réalité métier pour définir des règles d'alertes qui aient du sens. Une alerte déclenchée au mauvais seuil est inutile — voire dangereuse si elle génère trop de faux positifs que les opérateurs finissent par ignorer.

PARTIE CODE

PARTIE BUSINESS & ANALYSE

Tâches techniques

1. Coder les jobs Spark Streaming (lecture Kafka)

Créer `spark_streaming.py` qui lit en continu les topics Kafka avec `spark.readStream`. Parser les messages JSON selon le schéma défini par P1. Appliquer des fenêtres temporelles glissantes de 5 et 15 minutes pour calculer les moyennes mobiles par capteur et par zone.

2. Implémenter les règles d'alertes en PySpark

Traduire le cahier des règles métier en code : si $CO_2 > 800$ ppm pendant 5 minutes consécutives dans la même zone → alerte ROUGE. Utiliser les fonctions `when/otherwise` de PySpark pour catégoriser chaque alerte (VERTE, ORANGE, ROUGE) avec un message explicite.

3. Ecrire les alertes dans Kafka et InfluxDB

Les alertes générées sont écrites dans le topic `smartcity-alerts` (pour que P5 les affiche dans Grafana) et dans InfluxDB (pour les graphiques de fréquence d'alertes dans le temps). Utiliser `writeStream` avec `outputMode append` et des checkpoints pour la tolérance aux pannes.

4. Tester et valider les performances

Mesurer la latence bout-en-bout (de l'envoi du message à l'alerte dans Kafka). L'objectif est moins de 10 secondes. Documenter les résultats dans un fichier `PERFORMANCE.md` avec les métriques mesurées.

Tâches business & analyse

1. Définir les seuils critiques basés sur des normes réelles

S'appuyer sur les normes OMS (qualité de l'air), le code de la route (trafic), et les standards municipaux (déchets) pour fixer des seuils justifiables. Ex : OMS recommande $PM_{2.5} < 15 \mu g/m^3$ en moyenne annuelle, mais un pic horaire à $75 \mu g/m^3$ justifie une alerte. Ces chiffres donnent de la crédibilité au projet.

2. Concevoir les scénarios d'alertes (qui reçoit quoi)

Définir la matrice de notification : une alerte ROUGE pollution → notification immédiate au service environnement. Embouteillage critique → notification à la police municipale. Poubelle pleine → notification à la voirie. Formaliser dans un tableau de routage des alertes.

3. Rédiger le cahier des règles d'alertes intelligentes

Document de référence listant toutes les règles : condition déclenchante, durée minimale avant alerte, niveau de criticité, destinataire, condition de résolution. Ce document est validé avec P1 avant d'être codé.

Spark Streaming — Détection alertes pollution

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, avg, window, when

df = spark.readStream.format('kafka') \
    .option('kafka.bootstrap.servers', 'localhost:9092') \
    .option('subscribe', 'smartcity-pollution').load()

df_parsed = df.select(from_json(col('value').cast('string'),
    schema).alias('d')).select('d.*')

df_alerts = df_parsed.filter(
    (col('co2_ppm') > 800) | (col('pm25') > 100) | (col('air_quality_index') > 300)
).withColumn('niveau',
    when(col('air_quality_index') > 400, 'ROUGE')
    .when(col('air_quality_index') > 300, 'ORANGE')
    .otherwise('JAUNE')
)
```

Livrables attendus — P3

- Job spark_streaming.py fonctionnel et testé
- Alertes visibles dans le topic Kafka smartcity-alerts
- Métriques écrites dans InfluxDB (vérifiées par P5)
- Cahier des règles d'alertes (document PDF ou Markdown)
- Fichier PERFORMANCE.md avec latences mesurées

P4

Ingénieur Batch & Analyse historique

Batch Layer · Delta Lake · ETL · Tendances et recommandations

Présentation du rôle

P4 prend en charge la couche la plus analytique du projet : le Batch Layer. Son travail est moins spectaculaire que les alertes temps réel de P3, mais il est tout aussi essentiel. C'est grâce à lui qu'on peut répondre à des questions comme 'le mardi est-il systématiquement plus pollué ?' ou 'quel quartier consomme le plus d'énergie d'éclairage ?'. Il est aussi responsable de transformer ces insights en recommandations concrètes pour une vraie administration.

PARTIE CODE

PARTIE BUSINESS & ANALYSE

Tâches techniques

1. Configurer Azure Data Lake Gen2 (zones de stockage)

Organiser le Data Lake en trois zones : raw/ (données brutes JSON telles qu'envoyées par IoT Hub), processed/ (données nettoyées en Delta Lake), aggregated/ (résultats des calculs batch). Chaque zone a ses permissions d'accès configurées via Azure Active Directory.

2. Coder les jobs Spark Batch — ETL JSON vers Delta Lake

Lire les fichiers JSON bruts depuis raw/, appliquer les transformations (déduplication, validation de schéma, enrichissement avec la zone géographique), et écrire le résultat en format Delta Lake dans processed/. Le job doit être idempotent : relancé deux fois, il ne crée pas de doublons.

3. Créer les tables Delta partitionnées

Partitionner les tables par date_partition et zone pour optimiser les requêtes. Une requête 'données du quartier industriel cette semaine' ne doit lire que les partitions concernées, pas l'intégralité du Data Lake.

4. Planifier les jobs avec cron et monitorer

Configurer un cron job qui lance le batch ETL toutes les heures. Ajouter un système de logging : chaque exécution écrit son statut (succès/erreur, nombre de lignes traitées) dans un fichier batch_logs.csv.

Tâches business & analyse

1. Analyser les tendances des données simulées

Une fois les données accumulées sur 2-3 jours de simulation, mener une analyse : quelle zone a l'AQI le plus élevé ? A quelle heure le trafic est-il le plus dense ? Y a-t-il une corrélation entre trafic et pollution ? Ces analyses sont réalisées en SQL sur Delta Lake.

2. Produire un rapport d'analyse des données historiques

Document de 2-3 pages présentant les findings de l'analyse : graphiques (exports matplotlib), tableaux de chiffres clés, commentaires interprétatifs. Ce rapport devient une section du rapport final rédigé par P1.

3. Formuler des recommandations opérationnelles

A partir des tendances détectées, proposer des actions concrètes : 'Augmenter la fréquence de collecte des déchets dans la zone industrielle ' le mercredi matin', 'Ajuster les horaires de feux de circulation rue X ' entre 8h et 9h'. Ces recommandations montrent que les données servent à quelque chose.

Spark Batch — ETL et écriture Delta Lake

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import to_date, hour, col
from delta import configure_spark_with_delta_pip

spark = configure_spark_with_delta_pip(
    SparkSession.builder.appName('SmartCityBatch')
).getOrCreate()

df_raw = spark.read.json('abfss://raw@smartcitylake.dfs.core.windows.net/pollution/')

df_clean = df_raw.dropDuplicates(['timestamp', 'device_id']) \
    .dropna(subset=['co2_ppm']) \
    .withColumn('date_partition', to_date('timestamp')) \
    .withColumn('heure', hour('timestamp'))

df_clean.write.format('delta').mode('append') \
    .partitionBy('date_partition', 'zone') \
    .save('abfss://processed@smartcitylake.../delta/pollution/')
```

Livrables attendus — P4

- Data Lake structuré (zones raw / processed / aggregated)
- Job batch_etl.py fonctionnel et idempotent
- Tables Delta Lake accessibles en SQL (vérifiées par P5)
- Rapport d'analyse des tendances (2-3 pages avec graphiques)
- Liste de recommandations opérationnelles pour la mairie fictive

P5

Ingénieur Dashboard & Expérience utilisateur

Grafana · InfluxDB · UX opérateur · Alertes · Démo soutenance

Présentation du rôle

P5 est la vitrine du projet : son dashboard est ce que le jury verra en premier lors de la soutenance. Son travail consiste à rendre visible et compréhensible tout ce que P2, P3 et P4 ont produit. Il ne s'agit pas juste de 'faire des beaux graphiques' : P5 doit comprendre qui va regarder ce dashboard (un opérateur de nuit différent d'un directeur de service) et adapter l'information en conséquence. Il est aussi celui qui maîtrise l'ensemble du projet de bout en bout, car c'est lui qui anime la démo live.

PARTIE CODE

PARTIE BUSINESS & ANALYSE

Tâches techniques

1. Installer et configurer Grafana et InfluxDB via Docker

Ajouter InfluxDB et Grafana au docker-compose.yml. Créer la base de données InfluxDB 'smartcity' et configurer la source de données dans Grafana. Vérifier que Grafana reçoit bien les métriques écrites par P3 (test : afficher le premier panel simple avec une valeur en temps réel).

2. Créer tous les panels du dashboard

Panel 1 : carte géographique des capteurs actifs (plugin Geomap). Panel 2 : gauge AQI par zone (rouge/orange/vert selon seuils P3). Panel 3 : time series trafic et pollution en temps réel. Panel 4 : liste des alertes actives depuis Kafka. Panel 5 : heatmap hebdomadaire des pics de pollution (données batch P4). Panel 6 : taux de remplissage des poubelles par zone.

3. Connecter Grafana aux sources de données

Source 1 (temps réel) : InfluxDB → métriques P3. Source 2 (historique) : PostgreSQL ou requête Delta Lake → agrégats P4. Configurer les intervalles de rafraîchissement : 5 sec pour le temps réel, 1 min pour les données historiques.

4. Configurer le système d'alertes Grafana

Créer des règles d'alerte Grafana qui envoient une notification par email ou webhook quand AQI > 300 pendant plus de 10 minutes. Ces alertes Grafana sont complémentaires aux alertes Spark de P3 : elles portent sur des tendances longues, pas sur des pics instantanés.

Tâches business & analyse

1. Concevoir l'UX du dashboard selon les profils utilisateurs

Définir 3 profils : opérateur de nuit (besoin : voir les alertes critiques immédiatement, interface sobre), directeur de service (besoin : vue synthétique sur 7 jours, comparaisons entre zones), citoyen (besoin : qualité de l'air dans mon quartier aujourd'hui). Adapter la hiérarchie des informations en conséquence.

2. Préparer et animer la démo live

Scénariser la démo : lancer le simulateur, montrer les données arriver en temps réel sur le dashboard, déclencher manuellement un scénario de crise (augmenter le CO2 simulé au-dessus du seuil), montrer l'alerte apparaître. Répéter la démo au moins 3 fois avant la soutenance.

3. Rédiger le guide utilisateur du dashboard

Document de 1-2 pages destiné à un opérateur municipal non-technique : comment lire chaque panel, que faire quand une alerte se déclenche, comment filtrer par zone ou par période. Ce document illustre que le système est pensé pour de vraies personnes, pas juste pour des ingénieurs.

Ecriture InfluxDB depuis Python (pour intégration avec P3)

```
from influxdb_client import InfluxDBClient, Point
from kafka import KafkaConsumer
import json

client = InfluxDBClient(url='http://localhost:8086',
                        token='smartcity_token', org='smartcity')
write_api = client.write_api()

consumer = KafkaConsumer('smartcity-pollution',
                          bootstrap_servers='localhost:9092',
                          value_deserializer=lambda m: json.loads(m.decode('utf-8')))

for msg in consumer:
    d = msg.value
    point = Point('pollution') \
        .tag('zone', d['zone']) \
        .field('aqi', d['air_quality_index']) \
        .field('co2', d['co2_ppm'])
    write_api.write(bucket='smartcity', record=point)
```

Livrables attendus — P5

- Dashboard Grafana complet avec 6 panels fonctionnels
- Système d'alertes Grafana configuré (email ou webhook)
- docker-compose.yml mis à jour avec InfluxDB + Grafana
- Guide utilisateur du dashboard (1-2 pages)
- Démo live scénarisée et répétée pour la soutenance

9. Planning 4 semaines

	Semaine 1	Semaine 2	Semaine 3	Semaine 4
P1	Setup Azure + GitHub + schéma	Suivi + validation livrables	Suivi + validation	Rapport final + slides
P2	Simulateur + Docker Kafka	Bridge IoT Hub → Kafka	Tests de charge	—
P3	Setup Spark local	Jobs Streaming + alertes	Tests latence + InfluxDB	—
P4	Config Data Lake	ETL + Delta Lake	Jobs Batch + cron	Rapport analyse
P5	Grafana + InfluxDB Docker	Premiers panels	Dashboard complet + alertes	Démo live soutenance

Points de synchronisation obligatoires

- **Fin semaine 1** : P2 démontre Kafka qui reçoit des messages simulés. P1 valide le schéma JSON. Tout le monde peut commencer à coder.
- **Fin semaine 2** : P3 montre une alerte générée dans Kafka UI. P4 montre une table Delta créée. P5 montre Grafana opérationnel.
- **Fin semaine 3** : Démo interne complète : simulateur → Kafka → Spark → InfluxDB → Grafana. Tout le pipeline bout-en-bout tourne.
- **Semaine 4** : Soutenance. P5 anime la démo. P1 présente l'architecture et le rapport. Tous les membres répondent aux questions.

10. Livrables & Critères d'évaluation

Livrable	Responsable principal	Critère de succès
Infrastructure Azure	P1	IoT Hub, Data Lake et groupes créés et accessibles
Repo GitHub structuré	P1	Branches créées, README complet, commits réguliers
Simulateur IoT	P2	4 capteurs envoient des données réalistes en continu
Pipeline Kafka fonctionnel	P2	Messages visibles dans Kafka UI sur 5 topics
Jobs Spark Streaming	P3	Alertes générées en < 10 sec après un pic simulé
Cahier des règles d'alertes	P3	Seuils justifiés par des normes réelles (OMS...)
Jobs Spark Batch	P4	Tables Delta créées et requêtables en SQL
Rapport d'analyse	P4	Tendances identifiées avec graphiques et recommandations
Dashboard Grafana	P5	6 panels opérationnels, données rafraîchies en temps réel
Démo live soutenance	P5	Scénario de crise démontré de bout en bout sans erreur
Rapport final	P1	Document complet : archi, résultats, analyse, conclusion

Règles de collaboration

- Chaque membre fait un commit GitHub au minimum une fois par jour de travail, avec un message clair (ex : 'feat(streaming): ajout filtre seuil CO2').
- Aucun merge dans main sans pull request revue par P1. Les branches feature/ sont la zone de travail individuelle.
- Toute modification du schéma JSON des messages doit être soumise à P1 et communiquée à toute l'équipe avant d'être appliquée.
- Réunion de synchronisation chaque début de semaine (30 min) : chaque membre dit ce qu'il a fini, ce qui bloque, ce dont il a besoin.
- Si un membre est bloqué plus de 2 heures sur un problème technique, il en informe l'équipe immédiatement — personne ne reste bloqué seul.